

ML ACADEMY

SQL for Data & AI Interview Handbook

Detailed Questions, Explanations & Worked Examples for Top Tech Interviews

SQL Fundamentals • Joins • Windows • CTEs • Aggregations • Analytics Solved Queries • Business Problems • Experiments • Performance • Data Quality

www.learnmlacademy.com

Free Resource - Share with your network

How to get the most from this handbook

This PDF is designed for serious interview preparation. Each topic combines an interview-ready answer with deeper explanation, examples, diagrams, code, calculations or system trade-offs wherever they improve understanding.

Recommended approach: answer the question yourself first, then compare with the explanation and work through the example.

For coding and SQL questions, state assumptions, approach, complexity and edge cases before writing code. For ML/AI system design, discuss requirements, trade-offs, failure modes, monitoring and security instead of claiming one architecture is always best.

Section	Topics
1. Research basis and further reading	1
TOTAL COVERAGE	1

1. Grain, joins and ranking patterns

Most SQL mistakes are not syntax mistakes. They come from losing track of row grain, duplicate multiplication or tie policy.

SQL interview contract State the output grain before writing SQL. Then ask what rows can duplicate, which timestamp defines the event, whether unmatched entities must remain, and what tie policy is required. Correct syntax at the wrong grain is still wrong.

SOLVED PROBLEM

Latest event per user

Problem. Return exactly one latest event for each user with deterministic tie-breaking.

Reasoning. Rank events inside each user by event_time descending, then break timestamp ties using a stable unique key such as event_id.

Dry run For user 7 with events at 09:00/e2, 09:05/e1, 09:05/e3, ORDER BY time DESC, event_id DESC gives e3 row_number=1.

```
WITH ranked AS (  
  SELECT e.*,  
         ROW_NUMBER() OVER (  
           PARTITION BY user_id  
           ORDER BY event_time DESC, event_id DESC  
         ) AS rn  
  FROM events AS e  
)  
SELECT user_id, event_id, event_time, event_name  
FROM ranked  
WHERE rn = 1;
```

Complexity	Edge cases	Follow-up
Dominated by partitioned sort; commonly $O(n \log n)$, depending on physical plan/indexes.	Equal timestamps, users with no events, event_id tie policy, timestamp precision/timezone.	Include all users, returning NULL event fields when a user has never had an event.

SOLVED PROBLEM

Top product per category with ties

Problem. Find the product(s) with highest total revenue in every category.

Reasoning. Aggregate revenue first at category-product grain, then rank aggregated rows inside each category. Use RANK if ties should survive.

Dry run If electronics P1=1000, P2=1000, P3=600, RANK returns P1 and P2 as rank 1.

```

WITH revenue AS (
  SELECT category, product_id, SUM(amount) AS total_revenue
  FROM orders
  GROUP BY category, product_id
),
ranked AS (
  SELECT revenue.*,
         RANK() OVER (
           PARTITION BY category
           ORDER BY total_revenue DESC
         ) AS rnk
  FROM revenue
)
SELECT category, product_id, total_revenue
FROM ranked
WHERE rnk = 1;

```

Complexity	Edge cases	Follow-up
Group + per-category sort/rank.	Refund/cancel policy, currency, tie behavior, NULL amount.	Return exactly one winner using ROW_NUMBER with a deterministic secondary key.

SOLVED PROBLEM

Users with no purchases

Problem. Return all users with zero qualifying orders.

Reasoning. Use NOT EXISTS to express an anti-join. It avoids duplicate multiplication and directly asks whether any matching row exists.

Dry runA user survives only when the correlated subquery finds no row with the same user_id.

```

SELECT u.user_id, u.segment
FROM users AS u
WHERE NOT EXISTS (
  SELECT 1
  FROM orders AS o
  WHERE o.user_id = u.user_id
);

```

Complexity	Edge cases	Follow-up
With an index on orders.user_id, anti-lookups can be efficient; otherwise a scan/hash anti-join may be used.	Cancelled/zero-value orders, NULL keys, historical versus lifetime definition.	Return users with no purchase in the last 30 days.

2. Retention, sessions and gaps-and-islands

Product interviews frequently use time because time forces you to reason about grain, ordering and exact business definitions.

SOLVED PROBLEM2026 SIGNAL

Retention and product-metric SQL recur in current Google/Meta data-science interview preparation.

Day-1 retention by signup date

Problem. For each signup date, calculate the percentage of users who were active on the calendar day after signup.

Reasoning. Build one row per signed-up user, determine whether that user has any next-day event, then aggregate the binary retained flag at signup-date grain.

Dry run100 users sign up on Monday. 42 have at least one Tuesday event -> D1 retention = 0.42, regardless of how many Tuesday events each retained user generated.

```
WITH user_retention AS (
  SELECT
    u.user_id,
    CAST(u.signup_at AS DATE) AS signup_date,
    CASE WHEN EXISTS (
      SELECT 1
      FROM events e
      WHERE e.user_id = u.user_id
            AND CAST(e.event_time AS DATE)
              = CAST(u.signup_at AS DATE) + INTERVAL '1 day'
    ) THEN 1 ELSE 0 END AS retained_d1
  FROM users u
)
SELECT signup_date,
       COUNT(*) AS signups,
       AVG(1.0 * retained_d1) AS d1_retention
FROM user_retention
GROUP BY signup_date
ORDER BY signup_date;
```

Complexity	Edge cases	Follow-up
Depends on event indexing and date expression; one correlated lookup per user conceptually.	Timezone, bot events, duplicate events, whether signup day itself counts, dialect-specific date arithmetic.	Compute D7 retention with exact-day and rolling-window definitions.

SOLVED PROBLEM2026 SIGNAL

Sessionization is a high-value window-function pattern for product analytics interviews.

Sessionize events with a 30-minute inactivity gap

Problem. Assign a session number per user whenever the gap since the previous event is more than 30 minutes.

Reasoning. Use LAG to compare each event with the previous timestamp, flag session starts, then cumulative-sum the flags per user.

Dry runEvents 10:00,10:12,11:00 -> gaps 12m then 48m. Flags 1,0,1 -> cumulative sessions 1,1,2.

```

WITH gaps AS (
  SELECT e.*,
         LAG(event_time) OVER (
           PARTITION BY user_id
           ORDER BY event_time, event_id
         ) AS prev_time
  FROM events e
),
flags AS (
  SELECT gaps.*,
         CASE
           WHEN prev_time IS NULL
            OR event_time > prev_time + INTERVAL '30 minute'
           THEN 1 ELSE 0
         END AS new_session
  FROM gaps
)
SELECT flags.*,
       SUM(new_session) OVER (
         PARTITION BY user_id
         ORDER BY event_time, event_id
       ) AS session_id
FROM flags;

```

Complexity	Edge cases	Follow-up
Per-user sorting dominates.	Exactly 30 minutes, late events, timezone, simultaneous timestamps.	Return one row per session with start, end, duration and event count.

SOLVED PROBLEM

Consecutive active-day streak

Problem. Find each user's longest streak of consecutive active calendar days.

Reasoning. Deduplicate to user-day grain, number dates in order, and use date minus row_number as a stable 'island' key for consecutive sequences.

Dry run Days Jan1, Jan2, Jan3 have row numbers 1,2,3; day-rn is constant, so they form one island. Jan5 gets a different key.

```

WITH days AS (
  SELECT DISTINCT user_id, CAST(event_time AS DATE) AS day
  FROM events
),
numbered AS (
  SELECT user_id, day,
         ROW_NUMBER() OVER (
           PARTITION BY user_id ORDER BY day
         ) AS rn
  FROM days
),
islands AS (
  SELECT user_id, day,
         day - rn * INTERVAL '1 day' AS grp
  FROM numbered
),
streaks AS (
  SELECT user_id, grp, COUNT(*) AS streak_days
  FROM islands
  GROUP BY user_id, grp
)
SELECT user_id, MAX(streak_days) AS longest_streak
FROM streaks
GROUP BY user_id;

```

Complexity	Edge cases	Follow-up
Deduplication plus per-user sorting.	Multiple events same day, timezone, missing calendar days, date arithmetic syntax.	Return start/end dates for the longest streak and define a deterministic tie rule.

3. Rolling metrics, experiments and funnels

These questions test whether you can translate product language into exact row/timestamp semantics.

SOLVED PROBLEM2026 SIGNAL

Rolling-user metrics and windows are reported in current Google data-science interview loops.

Seven-day active users

Problem. For each calendar day, count distinct users active during that day and the previous six calendar days.

Reasoning. A COUNT(DISTINCT user_id) over a time range is not supported uniformly as a window in every SQL engine. A portable interview approach uses a date spine and joins user-day activity into each seven-day window.

Dry runFor report day Jan7, count distinct users with any activity from Jan1 through Jan7 inclusive.

```
WITH user_days AS (  
  SELECT DISTINCT  
    user_id,  
    CAST(event_time AS DATE) AS activity_day  
  FROM events  
)  
SELECT d.day,  
       COUNT(DISTINCT u.user_id) AS active_users_7d  
FROM calendar_days d  
LEFT JOIN user_days u  
  ON u.activity_day BETWEEN d.day - INTERVAL '6 day' AND d.day  
GROUP BY d.day  
ORDER BY d.day;
```

Complexity	Edge cases	Follow-up
Naive range join can be expensive; pre-aggregated user-day tables and partition pruning help.	Timezone, incomplete date spine, very large history, definition of 'active'.	Design a scalable incremental table for 7-day, 28-day and 90-day active users.

SOLVED PROBLEM2026 SIGNAL

A/B testing plus SQL is a recurring Google/Meta data-science interview combination.

Experiment conversion by variant

Problem. Given experiment_assignments(user_id, variant, assigned_at), calculate post-assignment purchase conversion for each variant.

Reasoning. Preserve one assignment row per user, detect at least one qualifying purchase after assignment, then aggregate the binary conversion flag at variant grain.

Dry runVariant A has 100 assigned users and 12 users with any qualifying purchase -> 12% conversion. Multiple orders by one user still count once.

```

WITH converted AS (
  SELECT
    a.user_id,
    a.variant,
    MAX(CASE
      WHEN o.order_time >= a.assigned_at THEN 1
      ELSE 0
    END) AS converted
  FROM experiment_assignments a
  LEFT JOIN orders o
    ON o.user_id = a.user_id
  GROUP BY a.user_id, a.variant
)
SELECT variant,
  COUNT(*) AS users,
  AVG(1.0 * converted) AS conversion_rate
FROM converted
GROUP BY variant;

```

Complexity	Edge cases	Follow-up
Join plus per-user aggregation.	Multiple assignments, pre-experiment orders, exposure vs assignment, conversion window, bots, sample-ratio mismatch.	Add a seven-day conversion window and compute uplift with confidence intervals outside or inside SQL.

SOLVED PROBLEM

Ordered funnel: view -> add_to_cart -> purchase

Problem. Count users who reached each stage in the required order.

Reasoning. Find the first view, then the first cart after that view, then first purchase after that cart. Do not simply count users who ever performed each event because that ignores order.

Dry run A purchase before the first cart does not satisfy the funnel; only the first purchase after a valid cart counts.

```

WITH first_view AS (
  SELECT user_id, MIN(event_time) AS viewed_at
  FROM events
  WHERE event_name = 'view'
  GROUP BY user_id
),
cart_after_view AS (
  SELECT v.user_id, v.viewed_at,
         MIN(e.event_time) AS cart_at
  FROM first_view v
  LEFT JOIN events e
    ON e.user_id = v.user_id
   AND e.event_name = 'add_to_cart'
   AND e.event_time > v.viewed_at
  GROUP BY v.user_id, v.viewed_at
),
purchase_after_cart AS (
  SELECT c.user_id, c.viewed_at, c.cart_at,
         MIN(e.event_time) AS purchased_at
  FROM cart_after_view c
  LEFT JOIN events e
    ON e.user_id = c.user_id
   AND e.event_name = 'purchase'
   AND c.cart_at IS NOT NULL
   AND e.event_time > c.cart_at
  GROUP BY c.user_id, c.viewed_at, c.cart_at
)
SELECT COUNT(*) AS viewed,
       SUM(CASE WHEN cart_at IS NOT NULL THEN 1 ELSE 0 END) AS carted,
       SUM(CASE WHEN purchased_at IS NOT NULL THEN 1 ELSE 0 END) AS purchased
FROM purchase_after_cart;

```

Complexity	Edge cases	Follow-up
Multiple grouped passes; physical cost depends on event volume and indexes/partitions.	Repeated cycles, session boundary, timeout window, event clock skew.	Compute stage-to-stage conversion rates and segment by acquisition channel.

4. Product growth and model-evaluation SQL

Data/AI roles often ask you to combine SQL mechanics with statistical or ML interpretation.

SOLVED PROBLEM2026 SIGNAL

Current product/data interviews often combine aggregation with window functions and interpretation.

Month-over-month active-user growth

Problem. Calculate monthly active users (MAU) and percentage change from the previous month.

Reasoning. Aggregate distinct active users per month, then apply LAG to the monthly result. Separate the aggregation stage from the comparison stage.

Dry runJan MAU=100, Feb=120 -> growth=(120-100)/100=20%.

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', event_time) AS month,  
           COUNT(DISTINCT user_id) AS mau  
    FROM events  
    GROUP BY DATE_TRUNC('month', event_time)  
)  
,  
with_prev AS (  
    SELECT month, mau,  
           LAG(mau) OVER (ORDER BY month) AS prev_mau  
    FROM monthly  
)  
SELECT month, mau,  
       CASE WHEN prev_mau > 0  
           THEN 100.0 * (mau - prev_mau) / prev_mau  
       END AS growth_pct  
FROM with_prev  
ORDER BY month;
```

Complexity	Edge cases	Follow-up
Distinct aggregation over events, then tiny monthly window.	Missing months need date spine, user timezone, definition of qualifying activity, prev_mau=0.	Segment growth by country while preserving the correct denominator.

SOLVED PROBLEM

Model precision and recall by user segment

Problem. For each segment, calculate labeled count, precision and recall for a binary model.

Reasoning. Join predictions to the user dimension once, then conditionally aggregate true positives, predicted positives and actual positives. Protect zero denominators.

Dry runSegment A: TP=40, predicted positives=50, actual positives=80 -> precision=.8, recall=.5.

```

SELECT
  u.segment,
  COUNT(*) AS labelled,
  SUM(CASE WHEN p.predicted_label=1 AND p.actual_label=1 THEN 1 ELSE 0 END) AS tp,
  SUM(CASE WHEN p.predicted_label=1 THEN 1 ELSE 0 END) AS predicted_pos,
  SUM(CASE WHEN p.actual_label=1 THEN 1 ELSE 0 END) AS actual_pos,
  1.0 * SUM(CASE WHEN p.predicted_label=1 AND p.actual_label=1 THEN 1 ELSE 0 END)
    / NULLIF(SUM(CASE WHEN p.predicted_label=1 THEN 1 ELSE 0 END),0) AS precision,
  1.0 * SUM(CASE WHEN p.predicted_label=1 AND p.actual_label=1 THEN 1 ELSE 0 END)
    / NULLIF(SUM(CASE WHEN p.actual_label=1 THEN 1 ELSE 0 END),0) AS recall
FROM predictions p
JOIN users u ON u.user_id = p.user_id
WHERE p.actual_label IS NOT NULL
GROUP BY u.segment;

```

Complexity	Edge cases	Follow-up
Join + grouped scan.	Delayed labels, duplicate user dimension rows, label definition, small-sample volatility.	Add minimum sample size and confidence intervals before alerting on a segment.

SOLVED PROBLEM

False-positive rate by segment

Problem. Calculate $FPR = FP / (FP + TN)$ for each segment.

Reasoning. Use conditional aggregation over actual-negative rows. Keep the denominator aligned to actual negatives, not all observations.

Dry runIf 100 actual negatives contain 8 predicted positives, $FPR=8/100=.08$. The number of actual positives does not enter the denominator.

```

SELECT
  u.segment,
  1.0 * SUM(CASE WHEN p.actual_label=0 AND p.predicted_label=1 THEN 1 ELSE 0 END)
    / NULLIF(SUM(CASE WHEN p.actual_label=0 THEN 1 ELSE 0 END),0) AS false_positive_rate
FROM predictions p
JOIN users u ON u.user_id=p.user_id
WHERE p.actual_label IS NOT NULL
GROUP BY u.segment;

```

Complexity	Edge cases	Follow-up
Join + grouped scan.	No negatives in a segment, delayed labels, repeated predictions for same decision.	Compare FPR and recall across segments at one global threshold versus segment-specific thresholds.

Deep-dive SQL tutorial: grain, windows, experiments and model metrics

This section teaches the reasoning behind common SQL interview patterns, with emphasis on correctness at the intended grain.

Study method Do not memorize definitions only. For each concept, explain the idea, work the example, name a failure mode, and state one trade-off. That pattern matches how stronger technical interviews are evaluated.

1. Start with grain

Before writing SQL, state what one row represents in each table and what one row should represent in the output. Many interview mistakes are grain mistakes: joining a user table to multiple event rows can multiply counts, and aggregating before the correct join can lose information.

A good verbal habit is “one row per user/day/product” before writing the query.

2. Logical query processing order

Conceptually, SQL processes FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, window functions/order details depending on engine, then ORDER BY/LIMIT. Understanding logical order explains why aliases may not be available in WHERE and why filtering before aggregation differs from HAVING.

3. Join types and duplicate multiplication

INNER JOIN keeps matches; LEFT JOIN keeps all left rows; FULL joins preserve both sides where supported. The biggest interview trap is many-to-many multiplication. Check key uniqueness and aggregate to the intended grain before joining when necessary.

Worked example / interview scenario

If users has one row/user and purchases has many rows/user, joining then COUNT(*) counts purchases, not users. COUNT(DISTINCT user_id) or pre-aggregation may be needed.

4. Window functions

Window functions compute values across related rows without collapsing them. ROW_NUMBER, RANK and DENSE_RANK solve top-N problems; LAG/LEAD compare neighboring rows; SUM/AVG over ordered frames compute running or rolling metrics.

Know the difference between PARTITION BY and ORDER BY, and be explicit about window frames for rolling calculations.

5. Latest row per entity

A robust pattern uses ROW_NUMBER() OVER (PARTITION BY entity ORDER BY timestamp DESC, tie_breaker DESC) and keeps row_number=1. The tie-breaker matters when timestamps can collide.

Engine-specific QUALIFY can simplify the syntax, but the conceptual pattern is portable.

Deep dive continued

6. Retention cohorts

Retention begins with a cohort definition such as signup date or first activity. Then compare later activity relative to the cohort anchor. Avoid mixing calendar-day retention with rolling 24-hour definitions.

Use distinct user counts at the intended grain and document timezone handling.

7. Sessionization and gaps-and-islands

Sessionization compares each event timestamp with the previous event using LAG. Mark a new session when the gap exceeds a threshold, then cumulatively sum the flags to create a session ID. Gaps-and-islands uses similar boundary detection to group consecutive ranges.

8. Funnels and ordered events

A funnel requires event order, not just presence. A user who purchased before an add-to-cart event should not count as completing an ordered funnel. Use conditional aggregation with timestamps, self-joins or window logic depending on complexity.

Clarify whether multiple funnels per user are possible and what attribution window applies.

9. Experiment SQL

Experiment analysis joins assignment to outcomes while preserving the randomization unit. Use the first valid assignment, avoid post-treatment filtering, and compute conversion or continuous metrics by variant.

Guard against users appearing in multiple variants and events occurring before assignment.

10. Rolling metrics

Rolling metrics require a defined time frame and grain. A 7-day active-user metric is not simply a 7-row moving average if dates are missing. Create a date spine when needed and count distinct active users over the intended calendar window.

Deep dive continued

11. SQL for model evaluation

Predictions joined with eventual labels can produce TP, FP, TN and FN counts by segment. From those counts compute precision, recall and false-positive rate. Check label availability windows so recent predictions with missing labels are not treated as negatives.

12. Performance and correctness

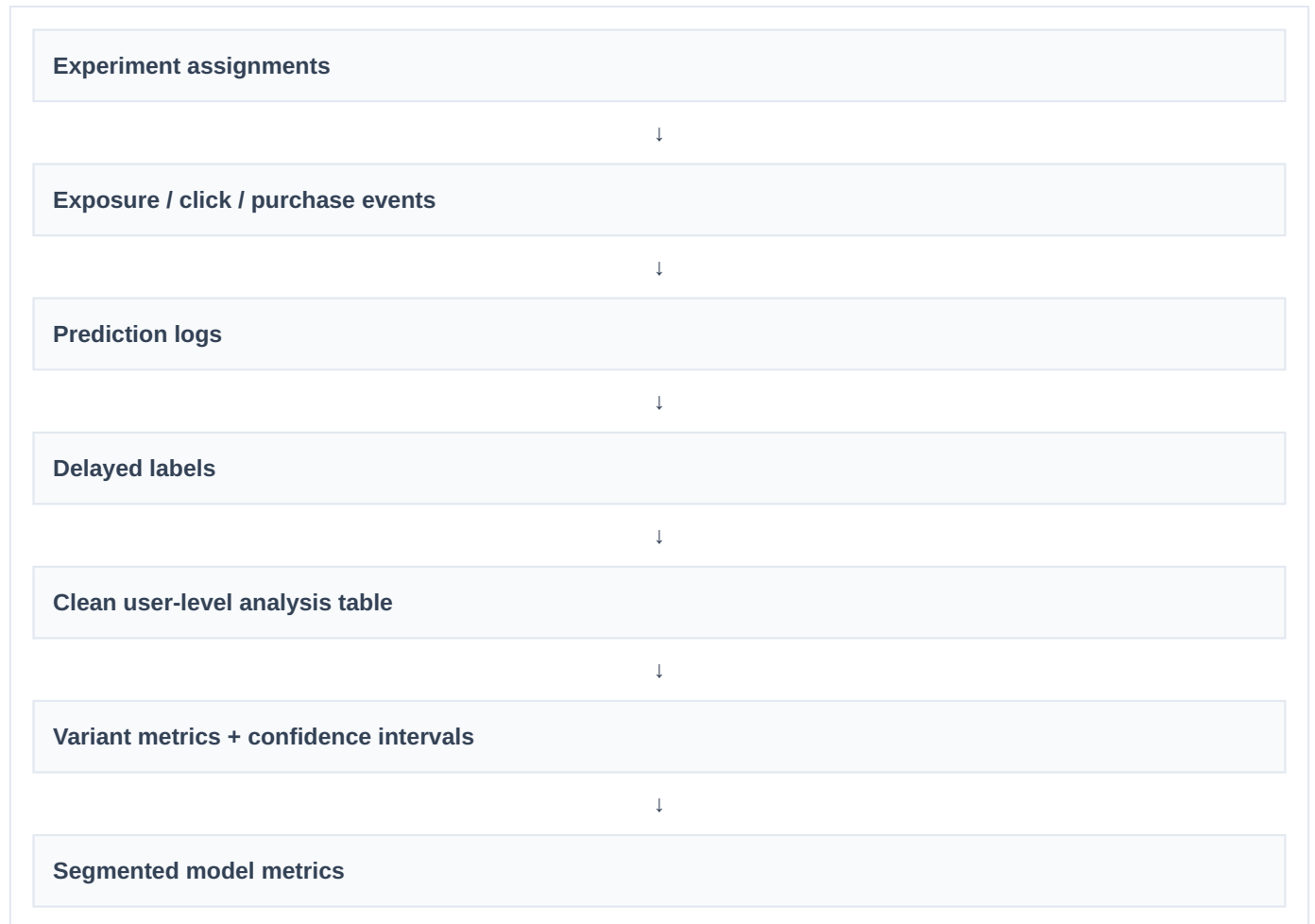
Indexes/partition pruning, selective filters and reduced scans improve performance, but correctness comes first. Avoid `SELECT DISTINCT` as a blind fix for duplicate joins. Inspect query plans for large production queries and reduce data early when it preserves semantics.

In interviews, explain both the logical result and what would matter at scale.

Complete case walkthrough

Case study - experiment and model evaluation in SQL

Case: compute experiment conversion and model-quality metrics for a recommendation launch.



Explain how you would choose one assignment per user, exclude pre-assignment outcomes, preserve the randomization unit, handle users with missing labels, and avoid duplicate joins between predictions and events.

Additional interview drill

Answer each question aloud in 60-90 seconds. Then add one example, one failure mode and one trade-off.

#	Question	Strong answer should mention
1	ROW_NUMBER vs RANK?	ties: row_number unique, rank gaps, dense_rank no gaps
2	WHERE vs HAVING?	row filter before grouping vs group filter after aggregation
3	Why can a join inflate counts?	many-to-many/many-to-one row multiplication
4	How get latest row?	partition + order desc + row_number=1
5	How define D1 retention?	cohort anchor and activity exactly one day later per definition
6	How sessionize?	lag timestamp, boundary flag, cumulative sum
7	What is a date spine?	complete calendar sequence for missing dates/rolling metrics
8	Why not DISTINCT blindly?	can hide grain/join bug
9	How compute precision in SQL?	$TP/(TP+FP)$ with safe division
10	What experiment pitfalls?	assignment integrity, pre-treatment events, unit mismatch
11	How top-N per group?	window rank partitioned by group
12	What should you clarify first?	table grain, keys, nulls, time zone, output grain

Hands-on labs and worked exercises

These exercises turn the concepts into things you can calculate, code, debug or design. Work them before reading the expected reasoning.

Lab 1 - Latest event per user

Goal Use window functions with deterministic ties.

Steps

1. Partition by user_id.
2. Order by event_ts DESC and event_id DESC.
3. Assign ROW_NUMBER.
4. Filter rn=1.

```
WITH x AS (  
  SELECT *, ROW_NUMBER() OVER (  
    PARTITION BY user_id ORDER BY event_ts DESC, event_id DESC  
  ) AS rn  
  FROM events  
)  
SELECT * FROM x WHERE rn=1;
```

Expected reasoning / result

A second sort key avoids nondeterministic results when timestamps tie.

Interview connection

State the input and output grain before writing SQL.

Lab 2 - Day-1 retention

Goal Build a cohort metric carefully.

Steps

1. Find each user signup_date.
2. Find distinct activity dates.
3. Join activity_date = signup_date + 1 day.
4. Divide retained users by cohort users.
5. Clarify timezone and calendar-day definition.

Expected reasoning / result

Retention definitions vary. A strong answer states whether D1 means next calendar day or 24-48 hours after signup.

Interview connection

Metric definitions are part of SQL correctness.

Lab 3 - Sessionize events

Goal Use LAG and cumulative sum.

Steps

1. Order events by user/time.
2. LAG previous timestamp.
3. Flag new session when gap >30 min or previous is null.
4. Cumulative SUM flags by user to create session_id.

Expected reasoning / result

This is a standard gaps-and-islands pattern.

Interview connection

Explain what happens with exactly 30 minutes and choose a rule.

Lab 4 - Experiment conversion

Goal Protect the randomization unit.

Steps

1. Select first valid assignment per user.
2. Join only outcomes after assignment.
3. Aggregate to one row per user.
4. Compute conversion by variant.
5. Check cross-variant contamination.

Expected reasoning / result

Do not count events as independent observations if randomization happened at user level.

Interview connection

Post-treatment filtering and duplicate assignments can invalidate experiment analysis.

Lab 5 - Precision/recall by segment

GoalEvaluate a model with SQL.

Steps

1. Join predictions to mature labels.
2. Create TP/FP/FN indicators.
3. Group by segment.
4. Use safe division for precision and recall.
5. Exclude predictions whose label window has not matured.

Expected reasoning / result

Delayed labels are a data-quality issue; missing recent labels should not be treated as negatives.

Interview connection

Connect SQL grain to model-evaluation correctness.

Last-minute revision sheet

Use this page in the final 30-60 minutes before an interview. Say each answer aloud in one or two sentences; if you cannot, revisit the relevant chapter.

Topic	One-line recall
Grain	Say what one output row represents before writing SQL.
ROW_NUMBER vs RANK	ROW_NUMBER forces one row; RANK preserves ties.
Retention	Count users, not events; define exact calendar/time window.
Sessionization	LAG gap -> new-session flag -> cumulative SUM.
Rolling users	Distinct users across calendar windows often need a date spine or pre-aggregation.
Experiments	Respect assignment/exposure time and one user per experimental unit.
Funnels	Enforce stage order, not just 'ever did event'.
ML metrics	Build numerators/denominators from conditional aggregates and guard zero denominators.

Research basis and further reading

This handbook is original LearnMLAcademy material. Public interview reports and official technical documentation were used to identify current topic emphasis and production practices. The explanations, examples, diagrams and code in this handbook are independently written.

InterviewQuery - Google Data Scientist Interview Guide (2026)

<https://www.interviewquery.com/guides/google-data-scientist>

Recent candidate reports emphasize SQL window functions, A/B testing, product metrics, ML fundamentals and behavioral rounds.

StrataScratch - Meta SQL Interview Questions (updated 2026)

<https://www.stratascratch.com/blog/facebook-meta-sql-interview-questions>

SQL questions framed as product and user-behavior problems rather than syntax drills.

Amazon Jobs - SDE II Interview Prep

<https://www.amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep>

Official guidance on syntactically correct code, edge cases, scalable solutions, Leadership Principles and STAR responses.

Google Machine Learning Crash Course

Official ML learning material emphasizes practical exercises, production systems, testing and monitoring in addition to model theory.

Made With ML MLOps Course

Production-oriented curriculum covering data, modeling, testing, reproducibility, serving, CI/CD and monitoring.

Current 2026 interview-preparation resources

Public guides increasingly emphasize production trade-offs, system design, debugging and role-specific reasoning rather than definition-only question banks.

Keep learning at learnmlacademy.com

167 in-depth lessons • Python code examples • Visual diagrams • Projects • Interview preparation • Generative AI • LLM & RAG • Agentic AI • Always free